

Hosting Model Interview Questions and Answers

Back to: [ASP.NET Core Web API Tutorials](#)

Hosting Model Interview Questions and Answers

In this post, we will discuss Hosting Model Interview Questions and Answers in ASP.NET Core. When preparing for interviews or working on real-world ASP.NET Core applications, one of the most confusing areas for students, freshers, and even experienced developers is **hosting models**. This confusion mainly comes from the fact that hosting behaviour differs depending on:

- **The Operating System (Windows vs Linux/macOS),**
- **The Web Server being used (IIS, IIS Express, Kestrel, Reverse Proxy), and**
- **The Deployment Mode (Self-Contained vs Framework-Dependent).**

Most people mix up terms like **In-Process**, **Out-Of-Process**, **Kestrel**, **SCD**, **FDD**, **EXE**, **DLL**, **dotnet host** because these concepts are explained differently across platforms. This set of **interview questions and answers** will clear up those confusions with simple, structured explanations and key points.

In Windows and IIS Server, do we have both options of hosting: In-Process and Out-Of-Process?

On Windows with IIS, ASP.NET Core supports both **In-Process** and **Out-Of-Process** hosting. In the **In-Process model**, your app runs directly inside the IIS worker process (w3wp.exe for IIS, iisexpress.exe for IIS Express), making it faster and more efficient. In the **Out-Of-Process model**, IIS acts as a reverse proxy to the Kestrel web server. Since ASP.NET Core 2.2, **In-Process is the default model** because of better performance, but you can still explicitly configure Out-Of-Process.

Key Points:

- Supported: Both In-Process and Out-Of-Process.
- In-Process = runs inside the IIS worker process (w3wp.exe / iisexpress.exe).
- Out-Of-Process = IIS forwards requests to Kestrel.
- Default since ASP.NET Core 2.2 = In-Process.

In Linux/Mac, do we have both options of hosting: In-Process and Out-Of-Process?

On Linux/macOS, there is no IIS, so the **IIS-specific hosting model concept** (In-Process vs Out-Of-Process) does not apply. Your app always runs on **Kestrel**, which is a standalone web server. Some Microsoft docs describe this

as “**Out-Of-Process**” to compare with **Windows IIS hosting**, but technically it is more correct to say: **Hosting Model is not applicable on Linux/macOS → app always runs directly on Kestrel.**

Key Points:

- Hosting Models = IIS-only concept.
- On Linux/Mac → IIS not available.
- App always runs on Kestrel directly.
- Hosting Model not applicable (but often described as Out-Of-Process).

If I run the application directly using Kestrel, is it ignoring the Hosting Models?

When you run with Kestrel directly, IIS is not involved. Since hosting models only exist when IIS is used, strictly speaking, **no hosting model is applied**. People often refer to this as “Equivalent to Out-Of-Process” for mental mapping purposes, as the app runs as its own process rather than being embedded in IIS. But the correct answer is: **No hosting model applies when running directly on Kestrel.**

Key Points:

- The hosting model applies only if IIS is involved.
- Running directly with Kestrel = no IIS → no hosting model.
- Strictly: IIS hosting model does not apply.
- Practically: Behaves like Out-Of-Process since Kestrel is a separate process.

When I publish the application with Self-Contained Deployment (SCD), why do I get both .DLL and .EXE?

In **SCD**, the .NET runtime and libraries are packaged with your app. The compiler produces both:

- a **Platform-Specific .exe (Windows)** or **Native Binary (Linux/Mac)** as the entry point, and
- your **main .dll** with IL code.

The .exe is a native entry point that launches your app using the bundled runtime and then calls into your .dll.

Key Points:

- Produces .exe (Windows) or native binary (Linux/Mac).
- .exe is the native starter (launcher).
- Produces .dll with app IL code.
- .exe bootstraps the .dll.
- Both files exist: DLL = app logic; EXE = entry point.

What does “.exe bootstraps the .dll” mean?

It means the .exe is just a **tiny native entry program** that runs first. Its job is to set up the .NET runtime and load the .dll into memory. Your actual business logic (controllers, services, middleware, etc.) lives in the .dll.

Key Points:

- .exe = native OS entry point.
- Initializes the .NET runtime.
- Loads the .dll into memory.
- Execution continues inside the .dll.

In Framework-Dependent Deployment (FDD), why do we get both .DLL and .EXE?

In FDD, your app depends on the runtime installed on the target machine. It produces a .dll (your app logic), and on Windows, also a small .exe that just calls dotnet <app>.dll. On Linux/macOS, no .exe is generated; you must run the .dll with dotnet.

Key Points:

- Depends on the installed .NET runtime.
- Produces .dll always.
- Produces .exe only on Windows (bootstrapper).
- Linux/Mac → run with dotnet <app>.dll.

When I build in Visual Studio, why do I get both .DLL and .EXE?

When building in VS (not publishing), you get a .dll as the main assembly. On Windows, a convenience .exe is also created, which just loads and runs the .dll via the runtime. On Linux/Mac, only the .dll is generated.

Key Points:

- Always generates .dll as main assembly.
- Windows builds also generate .exe (launcher).
- .exe calls into .dll.
- Non-Windows → only .dll.

In Windows, .EXE bootstraps .DLL. Then, in non-Windows, does the .DLL bootstrap itself?

No. The .dll never bootstraps itself. On Windows, the .exe launches the .dll. On Linux/Mac, the **dotnet host** plays the same role as the .exe, it bootstraps the .dll.

Key Points:

- Windows: .exe loads .dll.
- Linux/Mac: dotnet host loads .dll.
- .dll never bootstraps itself.
- The difference is only in **who starts first** (.exe vs. dotnet).

What is the .NET Host (dotnet host)?

The **.NET Host** is the small **bootstrap program** that is responsible for starting any .NET application.

- On your machine, this is the **dotnet executable** (called **dotnet.exe** on Windows, or just **dotnet** on Linux/macOS).
- It comes as part of the **.NET runtime and SDK installation**.
- Its main job is to **locate the right runtime**, load your app's .dll, and then pass control to the .NET runtime to actually execute your app's IL code.

Can I use dotnet to bootstrap the DLL in Windows?

Yes, on **Windows, we can also use dotnet to bootstrap the .dll**, exactly the same way we do on Yes. On Windows, you can run your app in two ways:

- Run the .exe directly, or
- Run dotnet MyApp.dll.

Both approaches work the same because the dotnet host is the global runtime bootstrapper.

Key Points:

- Windows:
 - Option 1: Run .exe (bootstrapper).
 - Option 2: Run dotnet MyApp.dll.
- Linux/Mac: only dotnet MyApp.dll.
- SCD: native .exe (Win/Linux/Mac) runs without dotnet.

If I run using Kestrel, IIS Express, or IIS, is it SCD or FDD?

Whether your application runs as **Self-Contained (SCD)** or **Framework-Dependent (FDD)** depends entirely on how you **published or built it**, not on whether you use Kestrel, IIS Express, or IIS. If you run directly from Visual Studio, it uses the system-installed .NET runtime, meaning it is **Framework-Dependent by default**.

- Depends on the publish type (SCD or FDD).
- Debug/Visual Studio → FDD (uses installed runtime).
- Publish as SCD → runs Self-Contained.
- Publish as FDD → runs Framework-Dependent.
- The hosting server (Kestrel/IIS) doesn't decide this.

Can I run the app using Kestrel only, or is a reverse proxy mandatory?

Yes, you can run your app directly on **Kestrel only**, on **all platforms (Windows, macOS, Linux)**. Kestrel is a full-fledged web server. However, in **Production Environments**, it is strongly recommended to place Kestrel **behind a Reverse Proxy** (IIS on Windows, Nginx/Apache on Linux) for added features like:

- Handling SSL certificates,
- Better security,
- Server Restarts,
- Load Balancing,
- URL Rewriting,
- Logging,
- Connection limits, etc.

So: **Reverse proxy is optional but recommended in production**. For **development and small internal apps**, running Kestrel alone is fine.

- Kestrel can run standalone on Windows, Mac, and Linux.
- A reverse proxy (such as IIS, Nginx, or Apache) is **not mandatory**.
- However, it is recommended for production due to its security, SSL, and performance benefits.
- For dev/local testing → Kestrel only is enough.

So, please be ready with the above IIS Hosting Model Interview Questions and Answers if you are attending any ASP.NET Core Interviews.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information